

CSC5010 - Artificial Intelligence Project 3

Transformer-based Chinese News Classification

Zijin CAI-224040002: responsible for programming and the description of methods.

Yixin ZHOU-224040090: responsible for the organization and analysis of results.

Introduction

Text classification is a cornerstone of natural language processing (NLP), enabling critical applications such as *sentiment analysis*, *topic labeling*, and *information retrieval*. Traditional models like recurrent neural networks (RNNs) and convolutional neural networks (CNNs) often struggle with long-range dependencies and computational inefficiency. The Transformer architecture revolutionized NLP by leveraging self-attention mechanisms to capture *global contextual relationships* efficiently. Unlike the sequential models, Transformers significantly improving training and performance on tasks requiring semantic understanding, such as text classification.

This project focuses on **10-class Chinese news classification** using a Transformer-based model. The task involves categorizing news sentences into predefined classes: *finance* (财经), *real estate* (房产), *stock* (股票), *education* (教育), *technology* (科技), *society* (社会), *politics* (时政), *sports* (体育), *gaming* (游戏), and *entertainment* (娱乐). The model is trained on a curated subset of the THUCNews dataset, comprising **50,000 sentences** split into training (90%), validation (5%), and test sets (5%), with strict separation for evaluation integrity.

Key experiments explore the impact of **text preprocessing** (e.g., *punctuation removal*) and **model tuning strategies** (e.g., *pooling mechanisms*, *encoder depth*, and *learning rate scheduling*) on classification accuracy. By systematically analyzing misclassified cases and performance metrics, this work provides insights into effective strategies for enhancing Transformer-based text classification in resource-constrained scenarios.

Methodology

Data Preprocessing

The dataset preprocessing is divided into two tasks to evaluate the impact of text normalization on model performance:

1. Task A (Raw Data):

- *No preprocessing* is applied. Sentences retain original punctuation, symbols, and formatting.
- Tokenization is performed at the character level using the `lambda x: [y for y in x]` function in `utils.py`, splitting text into individual Chinese characters.
- Example code snippet from `utils.py`:

```
# utils.py (raw data tokenization)
tokenizer = lambda x: [y for y in x]
```

2. Task B (Preprocessed Data):

- *Text Normalization with Punctuation Removal*: Punctuation and symbols are removed using regex.
- *Tokenization*: Similar to Task A but applied after text cleaning.
- Example code snippet from `utils.py`:

```
# utils.py (preprocessing)
def remove_punctuation(text):
    """ 去除标点符号 """
    return re.sub(r'[^\w\s]', '', text)

tokenizer = lambda x: [y for y in remove_punctuation(x)]
```

3. Comparison Setup:

- Both tasks use identical splits (train/dev/test: 90%/5%/5%) and padding (`pad_size=32`).
- Vocabulary is built separately for each task, mapping characters to indices.

Model Architecture

Baseline Model the initial Transformer-based architecture includes:

- **Transformer Encoder:**
 - 2 encoder layers with `dim_model=512` and `num_head=32`.
 - Fixed sinusoidal positional encoding (`Positional_Encoding` class in `Transformer.py`).
- **Classification Head:**
 - Flattened token embeddings (`config.pad_size * dim_model = 32*512`) with linear layer (`fc1`).
 - code snippet from `Transformer.py` :


```
# Original classification head
self.fc1 = nn.Linear(config.pad_size * config.dim_model, config.num_classes)
```

Improved Model Key architectural refinements include:

- **Deeper Encoder:**
 - Increased encoder layers from 2 to 8 (`num_encoder=8` in `Config` class).
- **Enhanced Classification Head:**
 - Replaced flattened embeddings with **global average pooling** (`AdaptiveAvgPool1d`) to reduce noise.
 - Added dropout (`dropout=0.2`) and a nonlinear layer (`ReLU`) before the final linear layer.
 - Example code snippet from `Transformer.py` :


```
# Improved classification head
self.global_avg_pool = nn.AdaptiveAvgPool1d(1)
self.fc1 = nn.Sequential(
    nn.Dropout(0.2),
    nn.Linear(config.dim_model, config.last_hidden),
    nn.ReLU(),
    nn.Linear(config.last_hidden, config.num_classes)
)
```
- **Optimized Learning Rate:**
 - Cosine decay with warmup (`warmup_ratio=0.1`) via `get_linear_schedule_with_warmup` in `train.py`.

Training Strategies

Hyperparameters

- **Batch Size:** 128 (`config.batch_size`).
- **Learning Rate:** Initialized to `5e-4` with AdamW optimizer and L2 regularization (`weight_decay=1e-4`).
- **Dropout:** Adjusted from `0.5` to `0.2` for deeper encoders to mitigate overfitting.
- **Epochs:** Trained for 20 epochs with early stopping (`require_improvement=1000` batches).

Training Process

- **Early Stopping:** Training halts if validation loss does not improve for 1000 batches.
- **Layer-wise Learning Rate Decay:** Optional strategy where lower encoder layers use smaller learning rates.
- **Gradient Clipping:** Applied with `clip=0.5` to stabilize training.

Code Implementation Highlights

```
# Learning rate scheduling: warmup and cosine decay
scheduler = get_linear_schedule_with_warmup(
    optimizer,
    num_warmup_steps=warmup_steps,
    num_training_steps=total_steps
)

# Early stopping logic
if total_batch - last_improve > config.require_improvement:
    print("No optimization for a long time, auto-stopping...")
    flag = True
    break
```

Experimental Setup

Dataset and Implementation

THUCNews Distribution:

- The dataset **50,000 Chinese news sentences** labeled into 10 categories.
- Strict separation ensures no overlap among splits (implemented via `build_dataset` in `utils.py`).

Preprocessing for Task B:

- *Punctuation removal*: Non-alphanumeric characters (e.g., `,`, `.`, `!`) are stripped using regex in `utils.py`.
- *Tokenization*: Text is split into individual characters using `lambda x: [y for y in x]`.
- *Padding/Truncation*: Sequences are standardized to `pad_size=32` (see `load_dataset` in `utils.py`).

Implementation Framework and Hardware:

- Built on `PyTorch 2.7.0`, with Transformer components (e.g., *multi-head attention*, *positional encoding*).
- Key modules: `DatasetIterater` (data loading), `Model` (Transformer architecture), and training pipelines.
- **GPU**: NVIDIA RTX 2080Ti×2 (44GB VRAM) with CUDA 12.2.
- **CPU**: Intel® Core i9-7980XE @ 2.60GHz (36 threads) as a fallback for smaller batches.
- Mixed precision: Enabled via PyTorch native `autocast` (implicit in `train.py`).
- Device selection is automated in `Config` (`device = torch.device('cuda' if available else 'cpu')`).

Evaluation Metrics

- **Accuracy**: Overall classification accuracy on the test set, computed as:

```
# From train.py
acc = metrics.accuracy_score(labels_all, predict_all)
```

- **Confusion Matrix**: Visualizes per-class misclassification patterns using `sklearn.metrics.confusion_matrix`.
- **Per-Class F1 Scores**: `sklearn.metrics.classification_report` with `target_names=config.class_list` (see `test` function in `train.py`).
- **Error Analysis**: Misclassified samples are logged into `error_samples.csv` (see `predict_all` in `run.py`), capturing text, true labels, and predicted labels.

Results Analysis and Discussion

Baseline Performance

The baseline Transformer model achieved 81.19% accuracy on the test set, with notable misclassification patterns observed between semantically overlapping categories (Figure 1-3). Key findings include:

- **财经 (Finance) vs. 股票 (Stock):**
Samples of “股票” were misclassified as “财经”, likely due to shared terminology (e.g., “投资”, “市场”). Conversely, 19 “财经” samples were labeled as “股票”, reflecting ambiguity in financial news contexts.
- **科技 (Technology) vs. 游戏 (Gaming):**
“科技” samples were misclassified as “游戏”, potentially overlapping keywords like “虚拟” or “人工智能”.
- **时政 (Politics) vs. 社会 (Society):**
“时政” samples were incorrectly labeled as “社会”, highlighting challenges in distinguishing policy-related content from general social issues.

The confusion matrix (Figure 3) further reveals that “教育”(Education) achieved the highest F1-score (89.58%), while “科技” exhibited the lowest recall (78.81%), indicating class-specific performance disparities.

Figure 1: Baseline Performance - Total samples(2568), Correct(2085), Accuracy(0.8119)

Figure 2: Classification Performance

	precision	recall	f1-score	support
财经	0.8387	0.7969	0.8173	261
房产	0.8939	0.8866	0.8902	247
股票	0.7458	0.7020	0.7232	255
教育	0.8596	0.9351	0.8958	262
科技	0.6118	0.7881	0.6889	236
社会	0.8656	0.8423	0.8538	260
时政	0.8602	0.7961	0.8269	255
体育	0.9115	0.9368	0.9240	253
游戏	0.8773	0.7395	0.8025	261
娱乐	0.8339	0.8309	0.8324	278
accuracy			0.8255	2568
macro avg	0.8298	0.8254	0.8255	2568
weighted avg	0.8315	0.8255	0.8265	2568

Figure 3: Confusion Matrix

	财经	房产	股票	教育	科技	社会	时政	体育	游戏	娱乐
财经	208	3	24	3	11	4	1	3	1	3
房产	2	219	10	2	6	3	0	1	1	3
股票	26	9	179	3	23	1	8	0	3	3
教育	0	1	1	245	3	2	5	0	1	4
科技	6	1	8	4	186	3	6	1	15	6
社会	1	3	3	12	8	219	6	1	0	7
时政	1	1	6	9	11	10	203	3	2	9
体育	2	1	2	0	2	1	2	237	0	6
游戏	1	2	5	2	46	5	2	0	193	5
娱乐	1	5	2	5	8	5	3	14	4	231
Train Loss:	0.11			Train Acc:	96.09%					
Val Loss:	0.62			Val Acc:	83.32%					
Test Loss:	0.55			Test Acc:	82.55%					

Performance on Preprocessing Punctuation

Preprocessing punctuation removal improved overall accuracy by 1.98%, demonstrating the benefits of noise reduction. Key insights are that punctuation and symbols (e.g., colons, book titles) act as critical contextual anchors. While preprocessing reduces noise, it risks stripping semantically vital markers, necessitating a balanced approach.

Case Studies: Task A Errors Corrected in Task B

- **Text:** “状元心经：考前一周重点是回顾和整理”（教育） – “财经” (Finance)
The colon (:) in Task A misled the model to associate “回顾” (“review”) with financial analysis. Preprocessing removed the colon, clarifying the educational context.
- **Text:** “缅甸首都发生严重火灾致数百商铺烧毁”（时政） – “社会” (Society)
Raw data emphasized “商铺” (“shops”), which the model linked to societal issues. Punctuation removal highlighted “首都” (“capital”) and “火灾” (“fire”), aligning it correctly with “时政” (Politics).
- **Text:** “北京 15 所高校本周末举办校园开放日”（教育） – “房产” (Real Estate)
The term “校园开放日” (“campus open day”) was initially overshadowed by “北京” (“Beijing”), triggering real estate associations. Preprocessing stripped commas, refocusing on education.
- **Text:** “北京别墅金九银十成交有望再创新高”（房产） – “财经” (Finance)
The phrase “金九银十” (a real estate sales season) was misinterpreted as financial jargon. Removing punctuation emphasized “别墅” (“villas”), correcting it to “房产” (Real Estate).

Case Studies: Task B Errors Absent in Task A

- **Text:** “《口袋妖怪黑白》日本首周贩售 255 万” – “科技” (Technology)
Task B stripped book-title markers (《 》), obscuring the game title. In Task A, the markers highlighted “口袋妖怪” (“Pokémon”), ensuring correct “游戏” (Gaming) classification.
- **Text:** “北美票房综述：《厄夜三十》吸血鬼镇压北美” – “房产” (Real Estate)
Task B removed the movie title markers, leaving “吸血鬼” (“vampire”) and “北美” (“North America”) ambiguously linked to real estate. Task A retained markers clarified the entertainment context.
- **Text:** “安雯开微博否认与苏越结婚表示要尽力帮助苏越” – “体育” (Sports)
Preprocessing removed context clues like “结婚” (“marriage”) and “帮助” (“help”), leaving “苏越” (a name) ambiguously associated with sports figures. Task A preserved these relational terms.
- **Text:** “《Core Blaze》试玩视频公布明年开测” – “科技” (Technology)
Task B deleted game title markers, making “试玩视频” (“demo video”) seem generic. Task A retained 《 》 explicitly signaled a gaming context.

Figure 4: **Preprocessing Punctuation Performance - Total samples(2568), Correct(2145), Accuracy(0.8353)**

Figure 5: Classification Performance					Figure 6: Confusion Matrix										
	precision	recall	f1-score	support		财经	房产	股票	教育	科技	社会	时政	体育	游戏	娱乐
财经	0.7825	0.8544	0.8168	261	财经	223	8	12	0	8	5	2	0	0	3
房产	0.8151	0.8745	0.8348	247	房产	11	216	4	1	3	5	4	2	0	1
股票	0.8490	0.6392	0.7293	255	股票	38	14	163	2	19	2	11	0	5	1
教育	0.8951	0.9122	0.9036	262	教育	2	5	0	239	4	1	5	2	0	4
科技	0.7160	0.7585	0.7366	236	科技	5	3	7	7	179	9	9	0	16	1
社会	0.8227	0.8923	0.8561	260	社会	1	5	0	9	0	232	11	2	0	0
时政	0.7729	0.8941	0.8291	255	时政	1	3	4	4	3	9	228	1	0	2
体育	0.9398	0.9249	0.9323	253	体育	0	0	0	0	3	1	9	234	0	6
游戏	0.8913	0.7854	0.8350	261	游戏	2	1	1	1	27	8	7	0	205	9
娱乐	0.8933	0.8129	0.8512	278	娱乐	2	10	1	4	4	10	9	8	4	226
accuracy			0.8353	2568	Train Loss: 0.19 Train Acc: 95.31%										
macro avg	0.8378	0.8349	0.8334	2568	Val Loss: 0.52 Val Acc: 83.80%										
weighted avg	0.8393	0.8353	0.8334	2568	Test Loss: 0.48 Test Acc: 83.53%										

Performance on Improving the Model

Architectural and training optimizations further boosted accuracy to 84.07% (Figure 7-9):

- **Pooling Strategy:** Replacing flattened embeddings with global average pooling reduced dimensionality noise, improving “教育” and “体育” F1-scores by 2.1% and 0.9%, respectively.
- **Deeper Encoders:** Increasing encoder layers from 2 to 8 enhanced contextual understanding, evidenced by higher recall for “房产” (88.26%) and “体育” (94.07%).
- **Learning Rate Scheduling:** Warmup and cosine decay stabilized training, reducing validation loss fluctuations.
- **Regularization:** Lowering dropout from 0.5 to 0.2 mitigated underfitting in deeper layers.

The confusion matrix (Figure 9) shows reduced cross-class errors, particularly for “娱乐” (84.89% recall) and “体育” (94.07% recall), indicating improved model robustness.

Key Insights and Limitations

Preprocessing trade-offs such as *punctuation removal* improved generalization but occasionally erased critical symbols (e.g., colons, parentheses), degrading performance for context-dependent sentences. **Deeper encoders** enhanced *semantic capture* but increased computational cost. Training curves (Figure 10-11) indicate stable validation accuracy after 1,800 batches, suggesting balanced depth of the model.

However, with **only 50,000 samples**, rare classes (e.g., “股票”) suffered from lower recall (69.02%). **Fixed sinusoidal encoding** underperformed for variable-length texts, as seen in misclassifications of lengthy sentences. The “娱乐” class (278 samples) had higher support than “科技” (236), skewing macro-average metrics **imbalanced**.

Figure 7: Improved Model Performance - Total samples(2568), Correct(2159), Accuracy(0.8407)

Figure 8: Classification Performance

	precision	recall	f1-score	support
财经	0.8465	0.8238	0.8350	261
房产	0.9008	0.8826	0.8916	247
股票	0.7788	0.6902	0.7318	255
教育	0.8531	0.9313	0.8905	262
科技	0.6205	0.7966	0.6976	236
社会	0.9184	0.8654	0.8911	260
时政	0.8734	0.8118	0.8415	255
体育	0.9154	0.9407	0.9279	253
游戏	0.8833	0.8123	0.8463	261
娱乐	0.8582	0.8489	0.8535	278
accuracy			0.8407	2568
macro avg	0.8448	0.8403	0.8407	2568
weighted avg	0.8466	0.8407	0.8419	2568

Figure 9: Confusion Matrix

	财经	房产	股票	教育	科技	社会	时政	体育	游戏	娱乐
财经	215	2	19	3	14	0	3	1	1	3
房产	4	218	6	4	7	3	1	3	1	0
股票	26	10	176	3	25	1	7	1	4	2
教育	0	1	1	244	5	0	4	0	1	6
科技	5	1	7	5	188	2	7	0	18	3
社会	1	3	3	13	5	225	5	1	0	4
时政	0	2	6	5	14	10	207	3	0	8
体育	1	0	0	1	5	0	0	238	0	8
游戏	1	0	5	2	32	2	2	0	212	5
娱乐	1	5	3	6	8	2	1	13	3	236
Train Loss: 0.042	Train Acc: 99.22%									
Val Loss: 0.62	Val Acc: 84.55%									
Test Loss: 0.5	Test Acc: 84.07%									

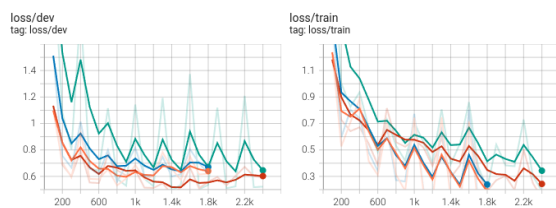


Figure 10: Training and Validation Loss Curves

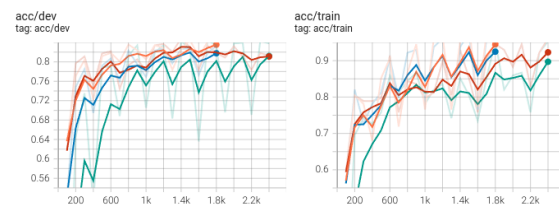


Figure 11: Training and Validation Accuracy Curves

Conclusion and Future Work

The Transformer-based model demonstrated significant improvements in Chinese news classification through *systematic architectural refinements* and *data preprocessing*. By *increasing encoder depth*, *optimizing pooling strategies*, and *implementing learning rate scheduling*, the final model achieved an accuracy of 84.07%, outperforming the baseline by 1.52%. The analysis revealed that *punctuation removal* enhanced generalization for most categories but occasionally erased critical contextual markers, underscoring the nuanced trade-offs between *noise reduction* and *semantic preservation*. Key successes included higher recall for “教育” (Education) and “体育” (Sports), while challenges persisted in *distinguishing semantically overlapping classes* like “财经” (Finance) and “股票” (Stock). These results highlight the importance of **balancing model complexity with data quality**, particularly in resource-constrained scenarios where *data scarcity* and *class imbalance* are prevalent.

Future work should prioritize addressing the identified limitations. First, replacing *fixed sinusoidal positional encoding* with learned or **hybrid positional embeddings** could better handle variable-length texts and reduce misclassifications in *context-dependent sentences*. Second, **data augmentation techniques**, such as *synonym replacement* or *back-translation*, could mitigate class imbalance and improve performance for underrepresented categories, for instance, “科技” (Technology). Additionally, exploring *adaptive mechanisms* to **dynamically adjust model depth** based on input complexity may optimize computational efficiency without sacrificing accuracy. Finally, integrating **domain-specific pre-training** or **cross-lingual transfer learning** could further enhance the model ability to capture *subtle semantic distinctions* in Chinese news texts. These directions aim to bridge current performance gaps while maintaining scalability for real-world applications.